Università degli Studi di Modena e Reggio Emilia

FACOLTÀ DI INGEGNERIA
Corso di Laurea in Ingegneria Informatica

Prova Finale

Creazione e sviluppo dell'applicazione Payper

Candidato:
Mazzi Samuele
Matricola 132389

Relatore:
Nicola Bicocchi

Anno Accademico 2020-2021

Indice

1 Introduzione **3**

1.1 Motivazioni 4

2 Strumenti utilizzati **6**

2.1 Flutter 6

2.2 Firebase 8

2.2.1 Realtime Database 8

2.2.2 Firestore Database 8

2.2.3 Firebase Cloud Messaging (FCM) 9

2.2.4 Cloud Functions 9

2.2.5 Cloud Storage 10

2.2.6 Authentication 10

2.2.7 Analytics 11

2.3 Altri strumenti 11

3 Panoramica generale **13**

3.1 Progetto iniziale 13

3.2 Estensioni 15

3.3 Il nucleo del progetto 16

3.4 Struttura del software 17

3.5 Presentazione grafica 19

4 Dettagli dell'implementazione 22

4.1 Hive 22

4.2 Struttura database 24

4.2.1 Realtime database 24

4.2.2 Firestore 25

4.2.3 Cloud Storage 27

4.3 Dettagli dell'invio scontrino 28

4.4 Gestione categorie 29

4.5 Gestione autenticazione 31

4.6 Gestione notifiche 32

5 API e altri strumenti 35

5.1 Google Cloud Functions (GCF) 35

5.2 Test 37

5.3 Sicurezza con Firebase Rules 39

6 Conclusione 41

6.1 Costi 41

6.2 Considerazioni finali 42

7 Requisiti funzionali 44

8 Ringraziamenti 49

Capitolo 1

Introduzione

Il progetto che verrà presentato è nato dall'idea di due ragazzi, Simone Leoni e Samuele Mazzi, il sottoscritto. Si tratta della creazione di un software che fosse in grado di evitare la stampa cartacea degli scontrini sostituendoli con scontrini digitali.
Gli scontrini, per legge [3], devono essere "stampati" ma non viene specificato con che mezzo, e anzi viene data la possibilità di inviarli in formato digitale. La legge che definisce questa regola è però relativamente recente e probabilmente è per questo che non esistono ancora idee simili sul mercato.
Per attuare questa idea la soluzione trovata è stata quella, da una parte, di creare un'applicazione *mobile* che fosse in grado di ricevere, grazie ad un codice identificativo, degli scontrini digitali e dall'altra parte di creare un "software di cassa" che potesse essere installato sui server/sulle casse di ogni negozio e che fosse in grado di inviare scontrini alla suddetta applicazione.
Nel procedimento di sviluppo va ricordato anche il supporto essenziale di due esperti nel settore che ci hanno permesso di capire a

fondo il funzionamento "dietro le quinte" dei sistemi di cassa nei supermercati.
Il progetto è nato nei mesi di Gennaio/Febbraio. Da lì è nata poi l'esigenza di un grafico e abbiamo quindi aggiunto nella squadra un altro ragazzo che studia allo IAAD, Gabriele Toselli, che ha quindi lavorato al *look and feel* dell'app insieme ad una più generale *brand identity*. Dopo diversi mesi di sviluppo e lavoro si è unito anche un altro ragazzo per aiutarmi nella programmazione e quasi in contemporanea si è unito a noi un altro ragazzo che si sarebbe poi occupato della parte di economia, Filippo Vidoni, ragazzo che studia Economia e fa già parte di una start-up che fattura più di mezzo milione di euro all'anno.
Il progetto è diventato quindi, a tutti gli effetti, un lavoro di team che si è rivelato utilissimo sia ai fini della startup ma anche e soprattutto a livello formativo-professionale.

1.1 Motivazioni

Un report del 2020 stilato da un'associazione americana [8] afferma che ogni anno, negli USA, si utilizzano fino a 3 milioni di alberi e quasi 3 miliardi e mezzo di litri d'acqua. Inoltre la produzione e la distribuzione di scontrini genera uno spreco equivalente a quello di 400,000 macchine per strada. E in tutto ciò, la maggior parte di questi scontrini vengono subito buttati o persi.
Non ci sono pervenuti dati sull'utilizzo a livello italiano ma è verosimile pensare che ci sia un utilizzo proporzionalmente elevato e che quindi anche nel nostro paese ci sia un alto spreco di risorse in

questo ambito.
Gli scontrini sono anche prodotti con sostanze tipo Bisfenolo-A (BPA) o Bisfenolo-S (BPS), interferenti endocrini che servono per stampare il testo sulla carta. A causa di queste sostanze non è possibile riciclare normalmente gli scontrini, come normali pezzi di carta, ma devono essere smaltiti insieme ai prodotti indifferenziati.
Inoltre quando tocchiamo gli scontrini, queste sostanze vengono in parte assorbite dal nostro corpo attraverso le mani. Sempre più studi stanno affermando che il BPA abbia impatti negativi sulla salute e sembrano esserci relazioni fra queste sostanze e problemi nello sviluppo del feto, lo sviluppo di diabate di tipo 2 e condizioni di tiroide.[1]
Qui di seguito un breve riassunto comparativo, sugli sprechi tra il 2019 e il 2020 (ovviamente in calo grazie al Covid-19, che ha ridotto il numero di acquisti in presenza):

	2019	2020
OVERALL VOLUME	280,000 metric tons	252,000 metric tons
WOOD USE	3,630,000 trees	3,270,000 trees
WATER CONSUMPTION	9,920,000,000 gallons (equivalent to over 7 million clothes washers operated/yr)	8,930,000,000 gallons (equivalent to over 6 million clothes washers operated/yr)
GREENHOUSE GAS EMISSIONS	5,110,000,000 pounds of CO2e (equivalent to 464,000 cars/yr)	4,600,000,000 of CO2e (equivalent to 418,000 cars/yr)
ENERGY USE	8,620,000 million BTUs (equivalent to over 10 million refrigerators operated/yr)	7,760,000 million BTUs (equivalent to over 9 million refrigerators operated/yr)
SOLID WASTE FROM PRODUCTION & DISPOSAL	330,000,000 pounds (equivalent to waste of 75 million people/day)	297,000,000 pounds (equivalent to waste of 67 million people/day)

*Per the advice of industry experts, our analysis included a 20 percent average of recycled content in thermal paper. We also used the supercalendered grade in the Paper Calculator, which is a category of printing & writing paper, which only sees 8 percent recycled fiber use globally. For this reason, it was determined that 20 percent would be a very conservative estimate.

Figura 1.1: Statistiche sull'impatto ambientale degli scontrini

Capitolo 2

Strumenti utilizzati

Per realizzare questo progetto mi sono servito di Flutter per sviluppare l'applicazione per Android e iOS, di Google Firebase per il back end e di altri strumenti per supportare lo sviluppo.

2.1 Flutter

Flutter è un framework open-source sviluppato da Google per la creazione di interfacce native per iOS e Android, e di recente permette anche la creazione di app per siti web, software Windows, Mac e Linux. Pubblicato nel maggio 2017, la prima release ufficiale avviene nel dicembre 2018. Alcuni esempi di utilizzo sono nelle app di: Google Ads, Alibaba, Groupon, Ebay [7]. Inoltre il 3 marzo 2021 è stato annunciato che Flutter sarà lo standard per lo sviluppo di applicazioni Ubuntu Firebase.
Flutter è basato sul linguaggio Dart, anch'esso sviluppato da Google, che permette di costruire interfacce grafiche per diverse piat-

taforme utilizzando sempre lo stesso codice. Il linguaggio Dart è un linguaggio di programmazione ad oggetti, molto simile a Java in diversi aspetti della sua sintassi.
Flutter si basa sul concetto che, a livello grafico, ogni elemento è un Widget. A livello pratico vuol dire che per creare un oggetto grafico basta creare una classe, estenderla con la superclasse Widget, e implementare un metodo di build in cui si dice cosa rappresentare. Ogni Widget è tipicamente l'insieme di altri Widget, per cui alla fine si viene a creare una struttura ad albero, formata da Widget Parent e Children.
Esistono due tipi di questi Widget, Stateful e Stateless, che si differenziano appunto per la presenza/assenza di uno stato interno. Modificare lo stato interno di un Widget vuol dire modificare i dati che vengono rappresentati a schermo, questo vuol dire che gli Stateless Widget, siccome non hanno uno stato, sono oggetti immutabili.
A livello pratico la gestione dello stato è relativamente semplice: si creano delle variabili nella classe e le si modifica a piacimento; nel momento poi in cui si vuole andare a rappresentare i dati modificati, si va a chiamare una funzione (della superlcasse Stateful Widget) chiamata setState(), con cui si fa un rendering del Widget corrente e di tutti i Widget sottostanti.
Piccola nota utile per capire gli estratti di codice: nel linguaggio Dart le variabili (e metodi) private non vengono definite con la keyword *private* ma diventano private se il loro nome inizia con il simbolo di underscore.

2.2 Firebase

È una piattaforma per la creazione di sviluppo per dispositivi mobili e web sviluppata da Google. In realtà era nata come compagnia indipendente nel 2011 ma è stata acquisita nel 2014 da Google. Nel nostro caso in realtà la vedremo "solo" per quello che riguarda il sistema di back end. Firebase implementa diverse e utilissime funzioni a questo pro. Da questa piattaforma si possono creare diversi progetti e ad ognuno si possono associare delle applicazioni Android, iOS e Web. Fra le principali funzioni proposte da Firebase ci sono: Realtime Database, Firestore, Cloud Storage, Cloud Functions, Messaging. Per questioni riguardanti alcune scelte nel progetto farò brevi riferimenti anche ai costi di ognuna di queste funzionalità.

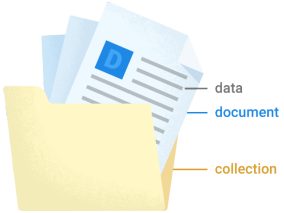
2.2.1 Realtime Database

Il primo tipo di database (NoSQL) sviluppato da Firebase, abbastanza primitivo ma potente, non permette la creazione di oggetti complicati sul database (o meglio, la permette ma non è comoda), siccome è strutturato come un immenso JSON, ma permette invece una connessione e una lettura/aggiornamento in realtime. Per quanto riguarda i prezzi, tende a costare molto di più (rispetto a Firestore) con l'aumentare delle dimensioni (1 euro/GB archiviato).

2.2.2 Firestore Database

Secondo tipo di database, anche questo NoSQL, è strutturato in documenti, i quali possono contenere Map di valori o collezioni, ov-

vero insiemi di altri documenti. Grazie a questa caratteristica sono permesse strutture dati molto più complesse rispetto al primo caso. I prezzi in questo caso sono molto più orientati verso l'utilizzo che verso l'archiviazione. Si parla di 0.18 euro/GB archiviati ma di 0.006 euro/10K letture e 0.018 euro/10K scritture. Per quanto possano sembrare numeri piccoli (e in effetti lo sono) per i nostri scopi arriveranno ad essere non trascurabili.



2.2.3 Firebase Cloud Messaging (FCM)

Questa funzionalità ci permette di inviare notifiche ad altri utenti. È totalmente gratuita e permette l'invio di notifiche a singoli utenti o a gruppi di utenti (raggrupati in topic) grazie all'utilizzo dei token univoci che definiscono il dispositivo utilizzato. Nel nostro caso verrà usato, molto banalmente, per notificare gli utenti dell'aggiunta degli scontrini e dell'aggiunta ad eventuali gruppi

2.2.4 Cloud Functions

Classico esempio di Function as a Service (FaaS), questo servizio permette di creare delle API, o anche funzioni eseguibili grazie a dei trigger, ovvero azioni di un determinato tipo negli altri servizi Google (per esempio, update/creazione/eliminazione di un documento su Firestore può essere un trigger per una determinata funzione su Cloud Functions). Tutte le funzioni possono essere scritte o in TypeScript o in JavaScript.

Nel nostro caso servirà da una parte, per esporre le API pubbliche alle casse che vorranno inviare gli scontrini e dall'altra agli utenti dell'app per eseguire azioni altrimenti impossibili (per esempio, inviare notifiche agli altri utenti è una funzionalità non supportata dal linguaggio Flutter ne da qualsiasi altro package).

2.2.5 Cloud Storage

Altro tipo di database che serve per contenere i file. Strutturato come una normalissima directory che quindi può contenere altre directory o con file con i relativi metadati. Nel nostro caso servirà per contenere gli scontrini dei singoli utenti.
Inoltre va fatto notare che i server vengono definiti "dual-region" o "multi-region" ovvero esistono più istanze dello stesso server localizzate in posti fisicamente diversi e lontani almeno 160km l'uno dall'altro [2], questo ovviamente per evitare che eventuali danni fisici ai server vadano a minare l'integrità di tutto il sistema.

2.2.6 Authentication

Sistema di autenticazione integrato di Firebase. Permette di eseguire login e registrazione in diverse modalità (email, Google, numero di telefono, Facebook, ecc...) e permette di assegnare un ID univoco ad ogni utente. L'autenticazione è di tipo persistente, cioè l'utente rimane autenticato fra una sessione e l'altra senza la necessità di dover eseguire continuamente il login.
Il sistema è quindi ovviamente in grado di dire se una mail è già stata utilizzata e se una determinata password è corretta per l'ac-

count a cui si sta provando ad accedere, e tutte queste funzionalità vengono integrate in una libreria già esistente.

2.2.7 Analytics

Questa funzionalità permette di raccogliere informazioni sull'utilizzo dell'app. Infatti dall'interno possiamo definire delle funzioni che ci notifichino quando avvengono determinate azioni. Utile sia per avere un riscontro statistico di quali parti dell'app vengono usate e in quale modo, ma utile anche e soprattutto perchè è grazie a questo che possiamo determinare quanti scontrini vengono scaricati e a quali negozi corrispondano.

2.3 Altri strumenti

Visto il lavoro di squadra è stato necessario anche l'utilizzo di altri software non direttamente collegati all'app ma che si sono dimostrati utili, se non essenziali, per lo sviluppo. Sembra quindi giusto spenderci due parole:

- **Figma**: Per l'interazione con il grafico della squadra è stato fondamentale l'utilizzo della web app "Figma" `https://www.figma.com` che consente di costruire l'interfaccia grafica di un'applicazione totalmente online, con un accesso condiviso a più utenti e la possibilità di commentare le diverse creazioni.

- **Github**: Il famosissimo servizio di hosting per progetti software, che permette di caricare online un progetto per poterlo

condividere con determinate persone (o direttamente al pubblico) ma soprattutto permette di tenere traccia delle varie versioni e aiuta uno sviluppo da parte di più sviluppatori sullo stesso software.

Capitolo 3

Panoramica generale

3.1 Progetto iniziale

Nella sua versione base, l'obiettivo dell'applicazione è quello di permettere agli utenti di ricevere gli scontrini sullo smartphone e di poterli conservare, di poterli inserire in categorie personali (spesa, sport, ecc. . .) o in categorie condivise cosi da poterli inviare in gruppi come famiglia o amici. Sia nei casi delle categorie personali che in quelle condivise è possibile vedere la spesa totale degli scontrini contenuti. La necessità di queste due aggiunte nasce per inserire nell'app una componenente di *utility*, perciò abbiamo aggiunto le categorie personali per aiutare una catalogazione degli scontrini semplice e intuitiva e le condivise per tutte quelle situazioni, per esempio famigliari, in cui una persona va a fare la spesa per conto di qualcun'altro e, se poi qualcuno volesse vedere quanto la famiglia ha speso, dovrebbe prendere tutti i telefoni e sommare solo certi scontrini. Invece così facendo, può avere direttamente tutto sotto mano, insieme ad una somma del costo totale per quel gruppo.

In generale comunque gli scontrini sono raggruppati per mese, e per ogni mese si può vedere la spesa complessiva.
Inoltre per incoraggiare l'utente in questa campagna green abbiamo pensato di tenere traccia della carta risparmiata dal singolo utente e da tutti gli utenti registrati. Siccome la quantità di scontrini necessaria per salvare un albero è eccessivamente alta, abbiamo pensato di mettere, come unità di misura, i bonsai. Perciò la barra di progresso per ogni utente progredirà ad ogni scontrini creato con l'applicazione e aumenterà di livello ogni volta che si sarà risparmiata la carta che si userebbe da un bonsai.
Grazie all'aiuto dei due professionisti che ci hanno accompagnato, abbiamo cambiato approccio al progetto non creando più un "software di cassa", ma una API pubblica a cui i server/le casse di ogni negozio potessero collegarsi e inviare gli scontrini.
Questa scelta è nata fondamentalmente per due motivi: il primo è uno scarico di responsabilità, se ci fossero stati dei problemi di invio dello scontrino saremmo stati subito chiamati noi, anche se il problema fosse stato, per esempio, la mancanza di connessione nel negozio. Invece così, prima di contattarci, i responsabili del negozio devono fare un controllo del sistema per verificare che tutto funzioni.
Il secondo motivo è l'elasticità con cui riescono ad essere esportati gli scontrini: infatti c'è quasi una totale indifferenza nel tipo di output digitale che si vuole dalla cassa (XML, JSON, txt, ecc...). Grazie a ciò è stato possibile creare una API che accettasse in input il formato JSON, conscio che ciò non avrebbe creato particolari impedimenti ai sistemi che avessero voluto collegarsi ad essa.

3.2 Estensioni

Successivamente a questa versione di base, sono state implementate altre funzionalità:

- **Aggiungere scontrini cartacei**: per aiutare un'iniziale "migrazione" dal sistema cartaceo a quello elettronico, è stato pensato di implementare una funzionalità già esistente in molte altre ma che poteva trovare un risvolto utile anche in questa. Ovviamente si considera un uso modesto di questa funzionalità, che dovrebbe tendere a diminuire nel corso del tempo.

- **Gestione carta fedeltà**: uno dei principali obiettivi che siamo arrivati ad avere è stato quello di riuscire a fare in modo che l'utente potesse usare l'app senza mai doverla tirare fuori... come? Con l'utilizzo di qualcosa che già un utente medio usa, ovvero la sua carta fedeltà. Perciò nell'app è possibile, con un metodo simile a quello di applicazioni già esistenti come *Stocard*, inserire/scansionare il codice della propria carta fedeltà cosicchè una volta passata alla cassa, questa possa essere utilizzata per identificare l'utente e inviare lo scontrino.

- **Creare pubblicità personalizzate**: una fra le più prossime aggiunte consisterà nel creare un sistema per mostrare all'utente pubblicità da parte dei negozi che frequenta più spesso e riguardanti solo i prodotti a cui è più interessato. Siccome ciò richiederà una certa collaborazione con le aziende e l'im-

plementazione è ancora in fase di sviluppo, non la descriverò oltre.

- **Esportazione rendiconto generale**: un'altra feature che verrà rilasciata a breve sarà la possibilità di esportare tutti i dati contenuti negli scontrini in un unico file pdf, così da poterlo condividere in qualsiasi modo si voglia.

3.3 Il nucleo del progetto

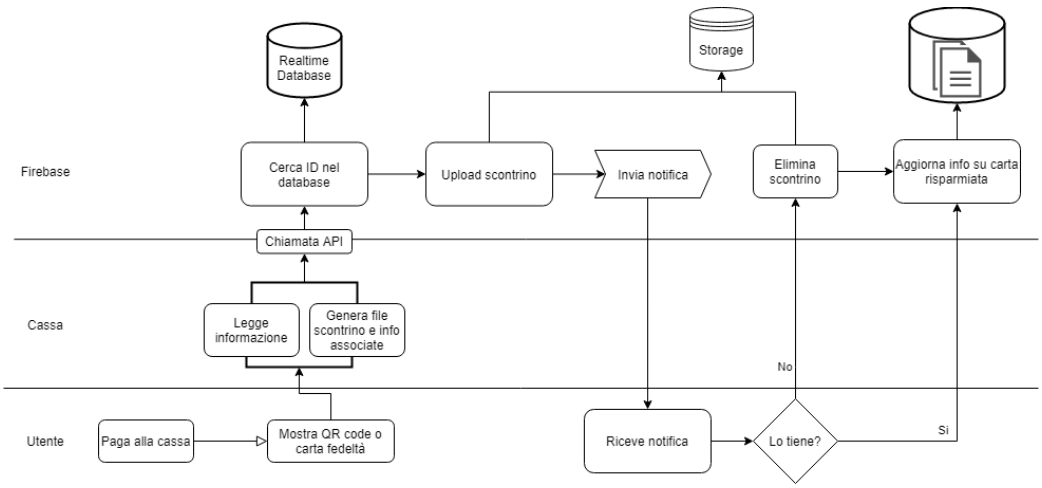


Figura 3.1: Activity diagram: ricezione dello scontrino

Questo activity diagram rappresenta il dialogo fra cassa, servizio di backend e applicazione, ad un livello di astrazione abbastanza elevato. Per capirci quindi si può vedere come il sistema chiama l'API remoto, passandogli lo scontrino formattato e le informazioni sull'utente a cui inviarlo (vedremo dopo in che modo). L'API una volta ricevuto lo scontrino esegue un parsing, estreando le informazioni che gli interessano, lo carica sullo Storage remoto e invia

una notifica all'utente interessato grazie a FCM Messaging.
A questo punto l'utente riceve la notifica dello scontrino e può decidere se mantenerlo o meno. Nel primo caso lo scontrino viene scaricato da dove era stato precedentemente caricato, così da essere salvato in locale e poi viene aggiunto un documento su Cloud Firestore, per tenere traccia dello scontrino con le sue informazioni principali. Nel secondo caso invece semplicemente il file viene eliminato dallo Storage remoto. In entrambi i casi viene aggiornato il valore riguardante il salvataggio di carta da parte dell'utente, visto che comunque ha evitato che venisse stampata della carta.
Per quanto riguarda l'upload e il conseguente download, all'inizio era stata implementata una versione in cui lo scontrino era direttamente incluso nella notifica, così da evitare una ridondante operazione di download. Ci si è poi resi conto che FCM impone un limite di dimensione massima di 4KB per notifica, che alcuni scontrini piuttosto lunghi superavano, e quindi è stato creato un sistema che frammentasse lo scontrino e lo inviasse in questo modo all'app. L'utilizzo in questo modo però risultava fragile e non conforme ad una classica mentalità cloud, inoltre se ci fosse stato qualche problema al client lo scontrino si sarebbe potuto perdere in via definitiva.

3.4 Struttura del software

Fin da subito, il design dell'applicazione si è dovuto scontrare con il problema del costo dei vari servizi Google Firebase. Infatti ognuno di questi ha un prezzo, ed era abbastanza prioritario ridurli al minimo. Inoltre si è palesata anche la necessità di creare un sistema

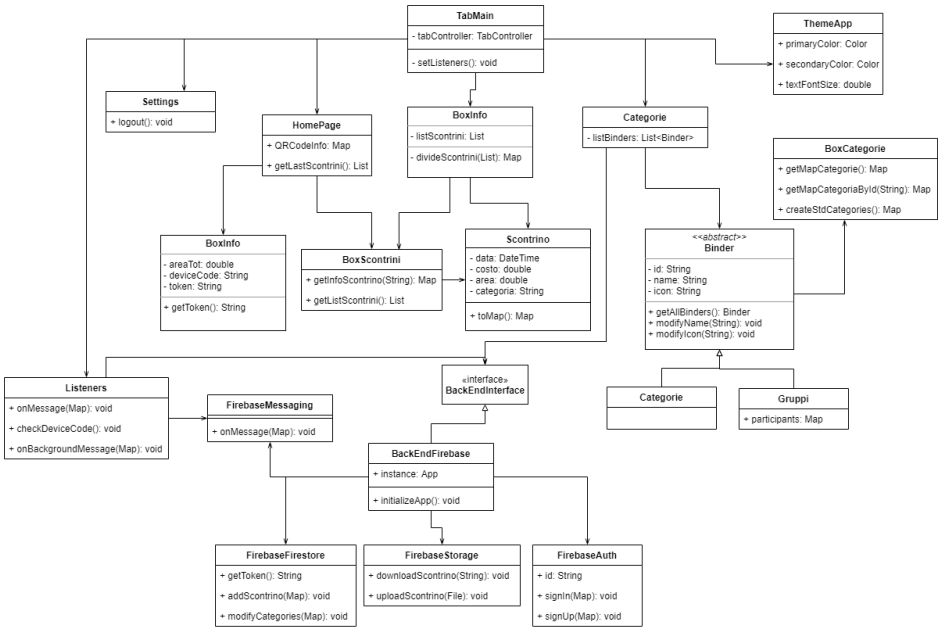


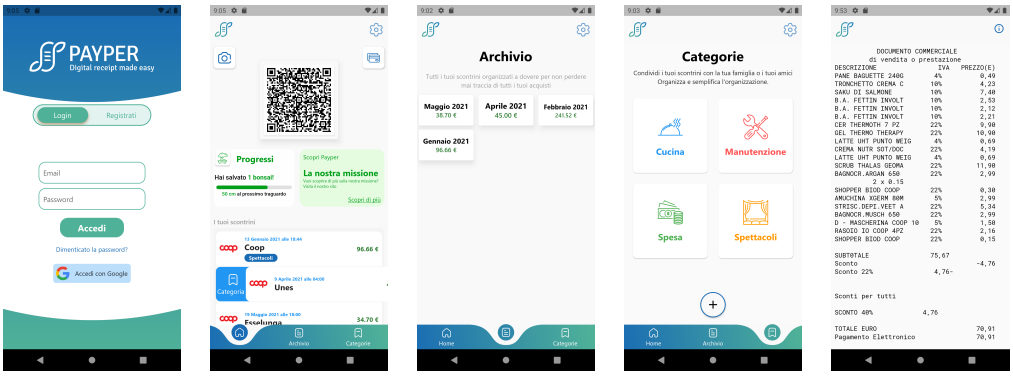
Figura 3.2: Class diagram generale

completamente cloud, siccome ormai gli utenti cambiano dispositivo molto spesso ed è quindi necessaria una soluzione che faccia persistere i dati da uno smartphone all'altro.
Questa premessa serve anche per giustificare un utilizzo, abbastanza massivo, di un sistema di "caching", in modo da ridurre al minimo i contatti con il servizio di backend, e quindi i costi. Approfondiremo dopo in che modo questo è stato implementato.
Come si vede da questo class diagram (che ovviamente rispetta la struttura del software ad un livello di astrazione abbastanza alto, data la complessità del codice) abbiamo una distinzione netta tra la parte di UI, quella di logica interna (business rules) e dialogo con il database.
La parte grafica verrà presentata meglio successivamente ma già da subito possiamo vedere che c'è una classe, TabMain, che imple-

menta le pagine principali, questo perchè sia in una struttura di tipo *Bottom tab bar*. Si può poi vedere come la logica di caching sia applicata in tutto il software e di come tutto giri intorno a dei dati locali. L'unica via attraverso cui avviene un aggiornamento da parte della componente cloud è grazie ai listener che "stanno in ascolto" di eventuali scontrini in arrivo, modifiche alle categorie condivise, o aggiornamenti simili.

3.5 Presentazione grafica

Per comprendere al meglio l'app effettueremo un brevissimo tour nelle pagine principali, cosicchè si abbia un'idea, anche visivamente più chiara dell'app. Come si potrà vedere, tolta ovviamente la pagina di login, tutte le altre pagine sono basate su una Tab bar view.



(a) Login (b) Home page (c) Archivio (d) Categorie (e) Scontrino

- **Registrazione**: questa è la prima e ovvia pagina di login/-registrazione a cui è stata aggiunta la possibilità di accedere direttamente con l'account Google. Per passare da una moda-

lità all'altra basta scorrere a destra o sinistra, oppure premere direttamente su una delle due voci associate.

- **Home page**: dopo aver effettuato la propria registrazione si accede alla home page. Qui è stata data maggiore importanza alla presenza del QR code, essendo il principale metodo di utilizzo per la ricezione degli scontrini. Vicino si possono trovare poi le icone per la gestione delle carte fedeltà e l'inserimento di uno scontrini cartaceo, ovvero gli altri due modi per aggiungere scontrini.
  Sotto questa sezione si può vedere la barra di progresso che dice quanti bonsai sono stati salvati dall'utilizzo, da parte del singolo utente, dell'app e quanti cm quadrati mancano per poterne salvare un altro. Premendo questa barra di progresso si accede poi ad una pagina che fa vedere l'animazione di questo bonsai che cresce a seconda dello stato di avanzamento dell'utente.
  Infine, ancora sotto la barra di progresso, c'è una piccola lista contenente gli ultimi scontrini aggiunti, per facilitarne la visualizzazione.

- **Archivio**: la pagina archivio contiene l'elenco di tutti gli scontrini, divisi per mese, con relativa spesa totale. Come si può vedere, da qua, ma anche nelle altre pagine, l'anteprima di ogni scontrino ha diverse caratteristiche. Oltre a rappresentare data, costo e negozio, permette di vedere subito la categoria in cui è stato aggiunto (grazie alla label apposita sotto il negozio).

- **Categorie**: questa pagina permette l'aggiunta di categorie pri-

vate e gruppi condivisi, per poter raggruppare gli scontrini in ”cartelle” private o condividere gli scontrini in gruppi contenenti contatti scelti dall'utente. La visualizzazione è aiutata dalla presenza di icone liberamente scelte dall'utente in un range limitato di possibilità. All'interno di ogni categoria sarà poi possibile vedere gli scontrini associati ad essa, il loro prezzo totale ed eventualmente anche i nomi degli altri partecipanti.

- **Pagina scontrino**: questa pagina rappresenta il core di tutta l'applicazione, per quanto semplice che sia, ma qui si può vedere come lo scontrino non cambi per nulla dalla sua versione cartacea: è infatti riprodotto fedelmente in ogni sua parte. In alto a destra sarà poi possibile vedere maggiori dettagli, riguardanti la data, il prezzo totale, categoria di appartenenza, ecc...

Capitolo 4

Dettagli dell'implementazione

Adesso analizzeremo meglio i punti più critici e importanti dell'applicazione.

4.1 Hive

Hive è un package creato per il linguaggio Dart, che permette di poter utilizzare un sistema di database locale ad accesso rapido, quasi come fosse una cache. I test parlano di tempi dell'ordine delle decine di millisecondi sia in lettura che in scrittura per 1000 iterazioni (test effettuati su Oneplus 6T with Android Q [6]).
Hive funziona tramite delle box, ovvero delle semplici cartelle, che si possono aprire e successivamente chiudere e che fungono da storage permanente e di veloce lettura/scrittura. Al loro interno, in modo nativo, si possono salvare i tipi di dati più famosi fra cui String, double, int, Map, List e pochi altri. Nel caso si volessero inserire

classi personalizzate ciò si può fare attraverso l'utilizzo di "Adapter" implementabili appositamente.
Ho quindi creato 4 box diverse:

1. **BoxInfo**: questa box contiene tutte le informazioni generali riguardanti l'utente, come il proprio username, il proprio token FCM, la quantità di carta risparmiata fino a quel momento, e altre informazioni di controllo.

2. **BoxGroups**: questa box contiene le informazioni riguardanti le categorie condivise a cui un utente appartiene. Sono quindi contenuti gli id di ogni categoria insieme al nome, l'icona scelta e l'elenco degli *uid* dei partecipanti con i corrispettivi username.

3. **BoxIndexScontrini**: questa box contiene i nomi di tutti gli scontrini ai quali è associata l'area di carta che avrebbero occupato, il costo totale dello scontrino, la data, l'eventuale categoria di appartenenza, se è stato ricevuto o inviato nel caso faccia parte di una categoria condivisa e se è stato correttamente scaricato dal server o se il file vero e proprio è ancora solo in cloud.

4. **BoxCategories**: questa box, analogamente alla seconda box, contiene tutte le informazioni riguardanti le varie categorie con i corrispettivi id, nome della categoria e immagine associata.

4.2 Struttura database

Per descrivere la struttura del database bisognerà dividere questo argomento in due parti, ovvero per i due tipi di database che sono stati usati (verrà presentato anche un terzo ma avrà un utilizzo scollegato dai primi). Infatti, fondamentalmente per motivi di costo, ma non solo, è stato scelto di utilizzare Realtime database quasi solo per contenere dati semplici e che sarebbero stati scritti/letti ripetutamente, e Firestore per tutti gli altri dati più complessi da organizzare, come quelli di ogni utente e di ogni categoria condivisa, e la cui la lettura/scrittura è più rara, anche se ovviamente non assente (un tradeoff necessario per lasciare un livello più alto possibile di manutenibilità).
Passiamo quindi ad analizzare le due soluzioni:

4.2.1 Realtime database

Come già detto questo database ha una struttura più semplice e vanta la gratuità di servizio a fronte del numero di letture/scritture. In compenso fa pagare per larghezza di banda utilizzata e la lettura di un nodo, forza anche la lettura di tutti i nodi figli. Quindi, per evitare costi elevati, la soluzione è creare tanti nodi, ma soprattutto, tante foglie, contenenti solo le informazioni essenziali, così da poter leggere solo il nodo che ci interessa e utilizzando una minima larghezza di banda.
Quindi qui ho salvato:

- **Area totale degli scontrini**: Siccome volevamo un contatore che tenesse traccia di tutta la carta risparmiata, da tutti gli

utenti, ho creato questa voce nel database, e l'ho inserita qua proprio perchè sarà letta e scritta con un ordine di grandezza *O(n)*, dove n sono gli scontrini creati, perciò si vedeva necessario l'utilizzo di una soluzione il cui prezzo non dipendesse dal numero di scritture/letture.

- **Fidelity cards**: Per l'estensione delle fidelity card è necessario salvare le fidelity card, oltre che nelle informazioni dell'utente, anche su questo database creando semplicemente una coppia chiave-valore, contenente come chiave il codice della carta fedeltà e come valore l'id dell'utente che la possiede. Questa coppia verrà letta da una delle API, che verranno chiamate dal software di cassa, con cui dalla carta fedeltà potrà risalire all'utente.
  Qui fa comodo sapere che Realtime database ha un limite di 200k connessioni simultanee al secondo per ogni database. Infatti con questo servizio si possono creare un numero a piacere di database, ognuno con le sue regole e la sua struttura. Perciò, per ora, per risolvere eventuali problemi di scalabilità che si potrebbero venire a creare nel caso di un utilizzo massiccio di questa feature, l'idea è di creare un database per ogni azienda che decida di collaborare con noi e che abbia un servizio di fidelity card, così ogni sua card verrà salvata lì.

4.2.2 Firestore

Vista la semplicità con cui è possibile gestire tanti elementi complessi, qui risiedono la maggior parte dei dati contenuti nell'app,

principalmente dati utente e dati riguardanti le categorie condivise.

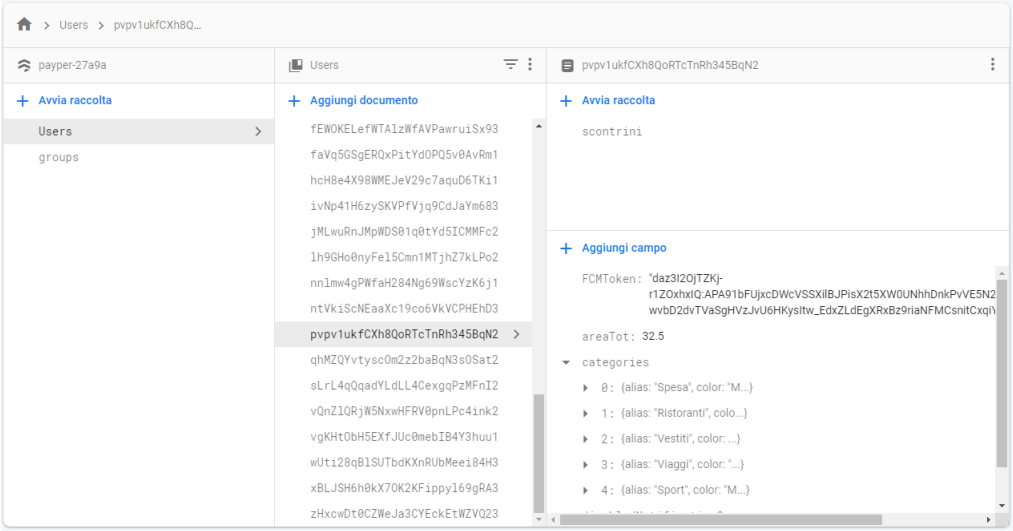


Figura 4.1: Esempio di documento utente in Firestore

- **Utenti**: Ovviamente è stata creato una collezione contenente un documento per ogni utente, identificato dal suo User ID (recuperato da Firebase Auth). Al suo interno sono stati inseriti i dati: FCM Token (cosicche possa essere recuperato dalla API al momento dell'invio dello scontrino), area totale risparmiata (fondamentalmente un dato derivato, uguale alla somma delle aree di tutti gli scontrini accumulati), una voce per tutte le categorie private dell'utente, con relative informazioni, una lista contenente i gruppi di cui fa parte l'utente e una lista contenente i gruppi da cui l'utente vuole ricevere notifiche.
  Inoltre questo documento contiene anche una collezione di scontrini, in cui è presente un documento per ogni scontrino: questo ha un nome uguale a quello con cui è salvato su

Cloud Storage, cosicchè possa essere facilmente trovato nel caso si voglia riscaricare, e contiene i dati principali, come area, costo, data e categoria di cui fa parte.

- **Categorie condivise**: Anche questa è una collezione contenente un documento per ogni gruppo esistente, con un ID generato automaticamente ad identificarli. All'interno di questi documenti ci sono i seguenti dati: il nome della categoria, l'immagine associata e una Map di partecipanti contenenti l'elenco degli utenti a cui, ognuno, è associato il suo username.

4.2.3 Cloud Storage

Come già scritto, questo tipo di database serve solo per contenere gli scontrini di ogni persona e la sua organizzazione è strutturata in banali directory. Quindi, la struttura gerarchica in questo caso consiste in una cartella "users" e in una sottocartella per ogni utente, al cui interno verranno inseriti i suoi scontrini.
Soppesando i pro e i contro, alla fine si è deciso inoltre di non creare cartelle separate per gli scontrini contenuti nelle categorie condivise, siccome questo avrebbe generato un ammontare di lavoro eccessivo per la loro corretta gestione (es. in caso di eliminazione di uno scontrino, non si può rimuovere dal Cloud Storage, altrimenti anche gli altri utenti lo perderebbero, ma in questo modo, quando l'utente farà il login su un altro dispositivo e si farà il fetching di tutti gli scontrini personali e condivisi, non sarà più possibile distinguere tra quali sono stati precedentemente cancellati e quali no, inoltre

come recuperare gli scontrini di categorie condivise di cui non si fa più parte?)

4.3 Dettagli dell'invio scontrino

Come prima cosa, il software di cassa legge l'ID dell'utente che intende salvare il suo scontrino sull'app. Già qui si presentano due opzioni: usare l'app o una carta fedeltà precedentemente registrata. Nel primo caso si farà vedere il QRCode, nel quale sono codificate in formato JSON le informazioni riguardanti ID utente e il suo token FCM, ovvero quello necessario per poter inviare la notifica; nel secondo caso, il codice della carta fedeltà verrà utilizzato per andare a cercare nel database l'associazione codice carta-ID utente.
Scelta la modalità di invio, il software di cassa, una volta costruito l'oggetto JSON da inviare, chiamerà l'apposita API con una chiamata POST. A questo punto l'API ha il compito di controllare la validità dei dati, andare a cercare l'ID utente nel caso sia stata passata una carta fedeltà, caricare il file dello scontrino nel Cloud Storage e inviare una notifica all'utente, contenente le informazioni principali riguardo allo scontrino (data, costo, area, ecc...).
Fatto ciò, l'app riceverà la notifica, che avvertirà l'utente dell'arrivo di uno scontrino e gli chiederà se intende tenerlo. A questo punto l'utente può decidere di rifiutare, eliminando lo scontrino anche dal Cloud Storage, oppure tenere lo scontrino ed eventualmente inserirlo subito in una categoria.
In contemporanea viene automaticamente creato uno scontrino "temporaneo" sul dispositivo: in questo modo se anche un utente can-

cella la notifica, o esce accidentalmente dall'alert di scelta, non avrà perso anche lo scontrino, infatti potrà toccarlo per vedersi rispuntare l'alert di scelta. In entrambi i casi, sia di scontrino tenuto o rifiutato, verrà aggiornato il contatore di carta risparmiata (siccome si avrà comunque evitato la stampa dello stesso).

4.4 Gestione categorie

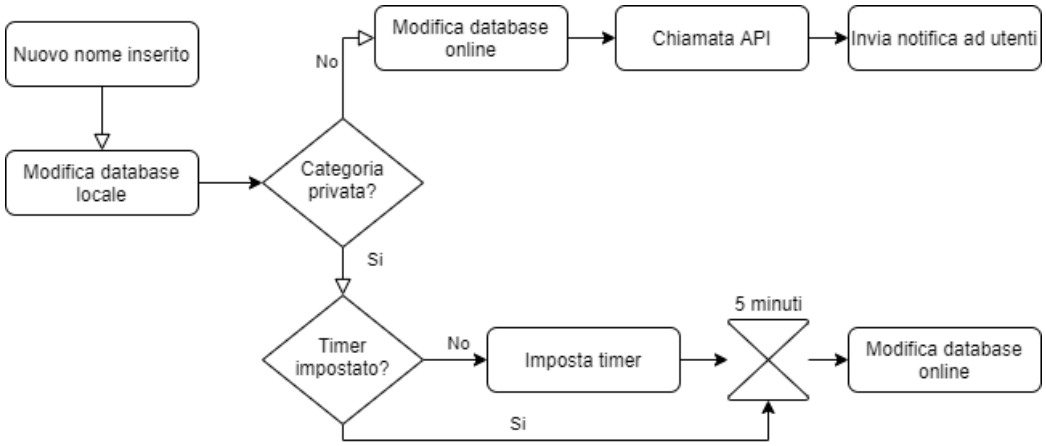


Figura 4.2: Actvity diagram per modificare il titolo di una categoria

Per le categorie condivise (o gruppi) e categorie private è stata creata una superclasse astratta che contenesse le proprietà comuni e alcuni metodi astratti (per esempio, creazione ed eliminazione dell'elemento). La differenza fondamentale fra i due è che i gruppi sono condivisi, quindi ogni modifica che viene fatta deve essere resa nota e disponibile anche agli altri partecipanti del gruppo invece la categorie sono personali e private.
Seppure in locale, i due elementi si somiglino (entrambi hanno un id, un nome, un immagine associata ed entrambi raccolgono scon-

trini) la loro implementazione in remoto risulta essere molto diversa. Infatti nel caso dei gruppi, siccome vogliamo che le informazioni siano consistenti fra i vari partecipanti del gruppo e siccome vogliamo che tutti questi possano accedervi dobbiamo creare un documento Firestore a parte in cui inserire queste informazioni e modificarlo ogni volta che viene fatta una modifica.
Qui va fatto notare che per questo procedimento non sono stati usati dei listener che ascoltano e aspettano per dei cambiamenti, siccome la spesa sarebbe stata molto elevata, ma, ad ogni cambiamento, l'utente invia, grazie alle Cloud Functions, una notifica a tutti i componenti del gruppo. La notifica non avrà corpo e non verrà vista dall'utente ma andrà a modificare direttamente i dati. Questo potrebbe esporre la funzionalità a certi problemi, anche solo di race condition, ma è stato valutato un rischio estremamente piccolo rispetto al guadagno.
Per le categorie private, invece, la gestione in remoto risulta decisamente diversa. Le categorie sono state infatti raggruppate in una Map che viene conservata nel documento Firestore dell'utente (si è scelta la Map perchè risultava il tipo di dato più adatto fra i pochi supportati dal sistema Firestore).
Se questa scelta è stata fatta per comodità è sicuramente stata fatta anche per il risparmio, grazie all'implementazione di un'altra strategia. Al momento della modifica di una categoria, non viene effettuata subito una scrittura sul database per mantenere la consistenza, ma viene fatto partire un Timer al cui termine verrà presa la Map locale contenente i dati sulle categorie e verrà caricata sul database al posto della precedente. Qual è la comodità? Siccome sta tutto

in una struttura dati se, mentre il timer è attivo, vengono fatte altre modifiche, alla fine del timer verranno caricate, con una sola scrittura, tutte le modifiche alle categorie. In poche parole questa strategia funziona da semplice buffer per eseguire più modifiche in un colpo solo.

4.5 Gestione autenticazione

Come già detto nell'introduzione, grazie a Firebase Authentication, molto del lavoro viene fatto automaticamente da Google. Quindi tutta la parte di conservazione dei dati autenticativi, la generazione di un ID utente, l'eventuale login con servizi di terze parti (Google, Facebook, ecc....) e la persistenza dei dati fra le sessioni è tutto fatto in automatico. Rimane solo un problema riguardante la logica dell'app così com'è strutturata ora: tutta la logica di caching si basa sul fatto che certe operazioni, non tutte ovviamente, avvengano prima in locale e poi debbano essere sincronizzate in cloud e non viceversa.
Per esempio, se modifico una categoria privata io prima faccio la modifica in locale e dopo la sincronizzo online, come già spiegato precedentemente. Cosa succederebbe quindi se io avessi due dispositivi diversi che possiedono lo stesso account? Avrei delle modifiche ai dati che non rispettano il mio flusso di controllo e questo romperebbe un po' tutta l'applicazione. Per evitare questo, al momento della registrazione viene salvato nel database una coppia chiave valore che associa l'ID utente con il suo device code, ovvero il codice univoco di cui tutti i dispositivi sono dotati, reperito grazie ad un

package esterno.
Fatto ciò, da quel momento in poi, ogni volta che l'utente accederà all'app, si attiverà questo listener che controllerà l'invarianza del codice. Questo perchè, nel caso un altro utente voglia accedere con quell'account da un altro dispositivo, potrà farlo, ma nel momento in cui accederà, modificherà il valore del device code associato al suo ID utente. Da un punto di vista utente si avrà un comportamento simile a quello di Whatsapp, per cui è possibile avere un solo dispositivo associato ad un solo account contemporaneamente. Ovviamente di tutto questo, l'utente viene messo al corrente, e anzi deve confermare di essere d'accordo.

4.6 Gestione notifiche

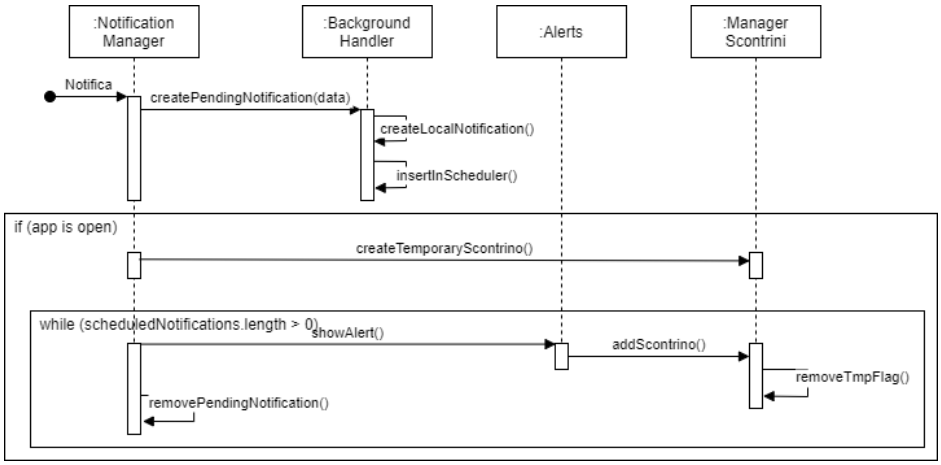


Figura 4.3: Sequence diagram per la gestione delle notifiche

Prima di entrare nel merito delle notifiche conviene spiegare meglio che meccanismo c'è sotto: è già stato detto che alla base del suo funzionamento c'è Firebase Cloud Messaging, ma come funziona?

Ad ogni utente su un dispositivo viene associato un token che rimane uguale salvo in alcuni casi come per esempio se viene eliminata la cache, o si disinstalla e reinstalla l'app [4]. Nel nostro caso poi il token viene salvato nel database, nel documento Firestore relativo all'utente che possiede questo token.
Quindi l'invio degli scontrini e anche di altre funzionalità basate su notifiche si basa sul raccoglimento del token associato all'utente a cui si vuole inviare la notifica. Altra potenzialità molto interessante del servizio FCM è che mette a disposizione l'iscrizione a dei topic: in poche parole i topic sono dei gruppi di token a cui si può dare un nome, e per cui se io invio un messaggio al topic è come se lo stessi inviando a tutti i partecipanti del gruppo, in una sorta di multicast; è comodo anche perchè la memorizzazione di tutti questi topic è gestita da Google.
Ora passiamo all'implementazione effettiva delle notifiche. Se da una parte la libreria esterna di Firebase Messaging mette a disposizioni funzioni molto comode (*onBackgroundMessage*, *onMessage*, *getInitialMessage*), che permettono di impostare dei listener sulle notifiche FCM sia in caso di app in foreground che in background, ciò presenta un paio di criticità. Normalmente infatti viene visualizzato un *alert dialog* se la notifica mentre l'app è aperta, o una notifica se l'app è in background: notifica su cui si potrà premere per far comparire la suddetta *alert dialog*. Cosa succede però se l'utente apre l'app non dalla notifica o anzi cancella notifica? Infatti mentre l'app è in background ogni azione eseguita, viene effettuata non su un altro thread, che quindi condividerebbe la memoria e lo spazio di indirizzamento utilizzati dall'applicazione fino a quel

momento, ma proprio in un altro processo. Questo diventa grave nel momento in cui arriva una notifica in background e noi non possiamo fare nulla che possa influenzare il funzionamento dell'applicazione quando verrà riaperta, e dobbiamo aspettare che l'utente chiuda completamente e poi riapra l'app per poter vedere le modifiche.
Bisognava quindi trovare una "memoria esterna" condivisa a cui entrambi i processi potessero far riferimento, e che fosse consistente. Questa memoria è stata trovata nello scheduling delle notifiche periodiche. Grazie ad un altro package infatti è stato fatto in modo che all'arrivo di una notifica, questa venisse inserita nello scheduler di notifiche periodiche (impostando il periodo ad una settimana) e poi, è stato creato un listener che notificasse la riapertura dell'app.
In questo modo, ogni volta che viene visualizzata una notifica o l'utente apre l'app, anche solo dal background, viene attivato un handler, grazie al listener precedentemente impostato, per controllare se esistono notifiche nello scheduler (che si riferisce solo alle notifiche proprie dell'app), nel caso ci siano mostrarle, e successivamente toglierle dallo scheduling. Nel sequence diagram si può vedere una funzione che crea uno scontrino temporaneo: serve per salvare lo scontrino in locale, indifferentemente dalla scelta dell'utente se tenerlo o meno: una volta deciso poi lo scontrino verrà reso "stabile" (e non più temporaneo) oppure verrà eliminato.
Tutto questo *work-around* a prima vista poco ortodosso e non formale si è dimostrato estremamente efficace, semplice e sicuro, nel senso che non vieta estensioni future, nel caso si volessero inserire altre notifiche periodiche ed esegue efficacemente il suo compito.

Capitolo 5

API e altri strumenti

5.1 Google Cloud Functions (GCF)

Ho già accennato prima all'utilizzo di Google Cloud Functions che ci permette di creare delle funzioni eseguibili con chiamate http, in poche parole delle API. Alcune di queste sono state create per un utilizzo interno all'app altre invece specificatamente per essere chiamate dall'esterno, dalle aziende che vorranno inviare gli scontrini.
Google mette fondamentalmente a disposizione macchine con capacità diverse, a seconda delle esigenze del cliente, con processori che vanno da 200 MHz a 1 GHz di clock, e memoria RAM associata che va da 128 MB a 1 GB di dimensione. Nel nostro caso, in quasi tutte le funzioni è stato scelto di usare le dimensioni minime, siccome le operazioni non risultano affatto onerose da un punto di vista computazionale.
Il costo di questo servizio si basa sulle caratteristiche della macchina scelte e sul tempo di esecuzione, calcolato in millisecondi: ri-

sulta quindi fondamentale cercare di ottimizzare ogni processo per migliorare al massimo le prestazioni.
Alla fine per le API pubbliche sono stati raggiunti i 400ms medi di esecuzione della funzione, sempre ignorando il fenomeno del *cold boot*: questo fenomeno infatti fa schizzare i tempi di esecuzione (si parla di 1500ms medi) siccome si verifica quando la funzione non viene chiamata per diverso tempo e quindi deve rieseguire tutto il file di esecuzione da capo, con import e inizializzazioni varie annesse.

```
205 exports.sendScontrinoStandard = functions.https.onRequest( async (req, res) => {
206   if(req.method === 'POST'){
207     const infoS = req.body;
208     let infoUser = await admin.firestore().collection(usersCollection).doc(infoS.uid).get();
209     if(infoUser.exists === false) {
210       return res.status(404).send({status: "Error", message: "Inexistent user"});
211     }
212     const token = infoUser.get(tokenFCM);
213     if(token === null) {
214       return res.status(404).send({status: "Error", message: "Inexistent token"});
215     }
216     if(typeof infoS[negozioScontrino] !== String || !Array.isArray(infoS[ricevutaScontrino])
217       || typeof infoS[dataScontrino] !== String) {
218       return res.status(400).send({status: "Error", message: "Wrong parameters"});
219     }
220     if(parseFloat(infoS[costoScontrino]) === NaN || parseFloat(infoS[areaScontrino]) === NaN) {
221       return res.status(400).send({status: "Error", message: "Wrong parameters"});
222     }
223     let payload = {
224       'data': {
225         date: infoS[dataScontrino],
226         negozio: infoS[negozioScontrino],
227         area: infoS[areaScontrino],
228         costo: infoS[costoScontrino],
229         type: "sendScontrino",
230         title: "Nuovo scontrino",
231         click_action: "FLUTTER_NOTIFICATION_CLICK",
232       },
233       'notification': {
234         title: "Nuovo scontrino",
235         body: "Hai ricevuto un nuovo scontrino!",
236       }
237     }
```

Figura 5.1: Cloud function per invio scontrini

Come si vede dall'esempio, per creare una funziona bisogna esportarla, in modo tale che sia eseguibile dall'esterno. Nel nostro caso la funzione è attivabile solo su *http request*: la funzione che si viene a generare ha due parametri, *req* e *res*, il primo per ricavare i dati in input e il secondo per gestire l'oggetto che andrà in output. Si può

vedere come vengano fatti dei controlli per verificare il corretto procedimento del metodo e come si possano ritornare degli errori con relativi codici e messaggi. La funzione poi continua con la creazione del payload del messaggio da inviare al dispositivo che riceverà lo scontrino e con altri procedimenti che non sono utili ai nostri scopi.

5.2 Test

Per uno sviluppo manutenibile del software sono stati ovviamente creati dei test, principalmente di tre tipi: *unit test*, *widget test* e *integration test*.
Gli unit test sono i classici test che si occupano di controllare il corretto funzionamento delle singole parti del software, invece i widget test sono quei test che vanno a verificare la corretta visualizzazione degli elementi, risultano quindi estremamente utili.
Gli integration test invece sono quelli che mettono insieme questi due tipi di test, per cui è possibile eseguire azioni e verificare che queste vengano eseguite sia a livello di codice che a livello grafico.
A causa della natura cloud di questa applicazione, eseguire certi test, soprattutto gli integration test, risulta particolarmente difficile: per questo si ricorre ad un espediente chiamato *Mock*. Grazie ad esso si possono creare delle implementazioni alternative di determinati metodi, per cui si possono creare delle false chiamate ai servizi di backend in modo tale da rimuovere molti problemi tecnici.
Sono stati creati ed eseguiti con successo, 20 unit test e 10 widget test, anche se in tutto questo bisogna considerare che gli altri componenti del gruppo sono attivi giornalmente come tester. Gli

unit test scritti riguardano principalmente i validatori dei testi, le meccaniche locali di gestione delle categorie e degli scontrini.Per i widget test invece, è stato creato fondamentalmente un test per ogni pagina presente, cosicchè fosse controllata la corretta visualizzazione statica.

I test riescono ad essere simulati grazie ad una libreria interna a Flutter, che funziona in modo simile a jUnit.

In queste immagini possiamo vedere come il file di test sia composto da un main, al cui interno sono presenti diversi ”group” ovvero raggruppamenti di test e al suo interno delle funzioni di test, identificabili da un nome a libera scelta del programmatore. Per ogni test semplicemente si fa in modo di ottenere un valore di ritorno e poi si confronta quel valore con quello che ci aspetteremmo. Sono possibili anche altre modalità di test, come per esempio l’esecuzione di una funziona e il controllo che questa lanci una determinata *Exception*.

```
4  void main() {
5    group("Validators", () {
6      String textValue = "value";
7      String? nullValue;
8      String emptyValue = "";
9      String numericValue = "12345";
10     String numericDotValue = "12345.43";
11     String? res;
12
13     _testNullAndEmpty(Function(String?) validator) {
14       String emptyRes = "Questo campo non può essere vuoto";
15       res = validator(nullValue);
16       expect(res, emptyRes);
17       res = validator(emptyValue);
18       expect(res, emptyRes);
19     }
20
21     test("Number", () async {
22       String textWithNumbers = textValue+numericValue;
23       _testNullAndEmpty(Validator.numberValidation);
24       res = Validator.numberValidation(numericValue);
25       expect(res, null);
26       res = Validator.numberValidation(numericDotValue);
27       expect(res, null);
28       res = Validator.numberValidation(textValue);
29       expect(res, "Inserisci un numero valido");
30       res = Validator.numberValidation(textWithNumbers);
31       expect(res, "Inserisci un numero valido");
32     });
```

(a) Unit test per validatori

```
21 void main() {
22   TestWidgetsFlutterBinding.ensureInitialized();
23
24   String email = "est@ee.it";
25   String username = "username";
26   String password = "123456";
27   String confPass = "123456";
28
29   setUpAll(() async {
30     await initHive();
31   });
32
33   group("Test sign page offline", () {
34     testWidgets("Pagina login statica", (WidgetTester tester) async {
35       await tester.pumpWidget(StartingPage(ScreenSignForm()));
36       expect(find.byType(FormTextInput), findsWidgets);
37       expect(find.text("Email"), findsWidgets);
38       expect(find.byType(ButtonGoogle), findsOneWidget);
39       expect(find.byKey(Key("Sign_button")), findsOneWidget);
40       expect(find.byType(ScreenSignForm), findsOneWidget);
41       expect(find.byType(MyHomePage), findsNothing);
42       expect(find.byKey(Key("Registrati_menu_button")), findsOneWidget);
43       expect(find.byKey(Key("Logo_payper")), findsOneWidget);
44     });
```

(b) Widget test per la pagina di registrazione

5 CAPITOLO 5. API E ALTRI STRUMENTI

5.3 Sicurezza con Firebase Rules

Dal punto di vista della sicurezza, Firebase permette di creare regole per l'utilizzo dei vari servizi, in particolare quelli riguardanti i due database e il servizio di Storage. Queste regole permettono di controllare gli accessi in lettura e scrittura e di validare i nuovi dati. Per scrivere queste regole si utilizza una particolare sintassi per cui si dichiara in quale percorso del database/storage ci si sta riferendo e lì si definiscono le regole appropriate: Firebase controllerà le regole nella gerarchia e riterrà valida solo quella al livello più profondo; in mancanza di regole, vieta ogni tipo di operazione. Per esempio, per permettere la lettura delle macchine di un utente, scriverò:

```
match /users/cars/{car} {
    allow read: if true;
}
```

La dicitura fra parentesi graffe funziona da *wildcard* per indicare un qualsiasi elemento nella "cartella". Dopo la keyword **if** va inserita la condizione per cui può avvenire l'azione: in questo caso è stata semplicemente resa sempre vera. Quindi, questo pezzo di codice mi permetterà di leggere le macchine degli utenti, ma nessun altro tipo di dato.
Perciò, usando questa procedura in modo più complesso, sono stati protetti tutti i dati in modo tale che ci fosse un accesso più ristretto possibile ad ognuno di essi, a volte con regole semplici come per esempio il controllo che il nome della collezione Firestore che si vuole andare a modificare sia lo stesso del proprio ID e altre volte con controlli più sofisticati come per esempio la verifica che l'id della

categoria condivisa che si vuole modificare sia nell'elenco delle categorie di cui l'utente fa parte.
Ovviamente questo non basta: un tipo di vulnerabilità a cui il sistema è tutt'ora scoperto sono gli attacchi di tipo DDoS, ma presto verrà provveduto anche a questo utilizzando un sistema di protezione Google, ovvero Cloud Armor, che permette di utilizzare un sistema di adaptive protection [5], il quale impara in modo statistico il normale andamento del traffico di rete e quindi rileva eventuali anomalie, bloccandole.

Capitolo 6

Conclusione

6.1 Costi

Concluso il discorso della progettazione software e della sua implementazione credo sia doveroso far vedere come l'obiettivo di un risparmio massimo sia stato rispettato abbondamente. Nel merito del costo per scontrino, di seguito potete vedere le tendenze di prezzo dei singoli componenti (quelli tratteggiati, ovviamente solo di backend) e la tendenza "globale", la retta intera: sull'ascissa il numero di scontrini prodotti e sull'ordinata la spesa ovviamente in euro. Senza andare nei dettagli, per motivi di privacy, si può vedere come l'aumento dei costi sia lineare rispetto al numero di scontrini e volendo guardare ai numeri, alla fine siamo riusciti ad ottenere un costo, per scontrino, circa 300 volte più economico della produzione di uno scontrino cartaceo.
Bisognerebbe valutare anche i costi relativi al semplice utilizzo dell'app da parte del singolo utente; non andremo a trattarlo qui però, perchè non è un prezzo così incidente sul totale, e si basa fonda-

mentalmente su stime di utilizzo mensili, quindi anche i numeri sarebbero delle previsioni potenzialmente sbagliate.

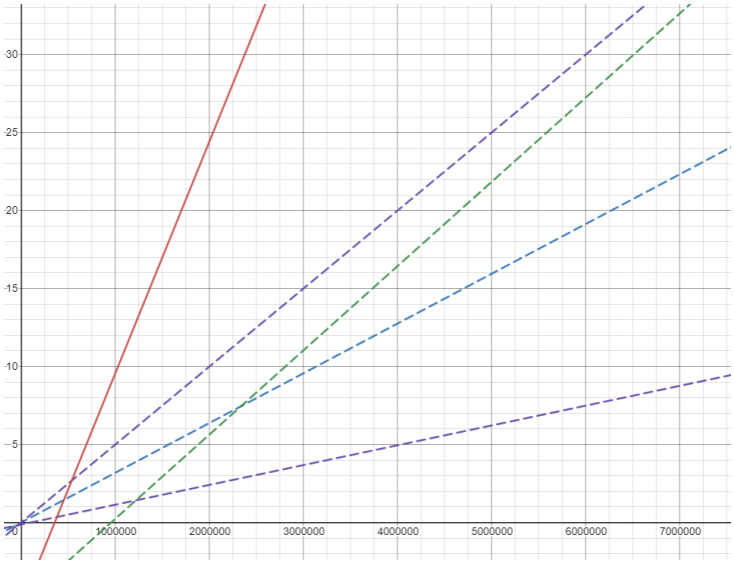


Figura 6.1: Costo degli scontrini

6.2 Considerazioni finali

Il progetto così strutturato si propone quindi di essere una valida e stabile alternativa all'utilizzo attuale degli scontrini. Mi rendo ovviamente conto che il lavoro svolto è lungi dall'essere professionale o industriale, ma d'altro canto, il progetto è stato eseguito tenendo bene a mente i concetti imparati nel corso di "Ingegneria del software" e comunque, anche adesso, sono in corso lavori continui per approntare miglioramenti del software così da avvicinarmi sempre di più a questo standard. Inoltre l'ambiente estremamente innovativo e il periodo storico in cui ci troviamo prolifera di start-up e giovani che riescono a farsi strada nel mondo del lavoro con nuove

idee, quindi ho fatto semplicemente il possibile per non essere da meno con un software la cui difficoltà di realizzazione non sembrava, e non è stata, eccessiva.
La start-up è stata fondata ed ora ci stiamo mettendo in contatto con varie piccole aziende sparse in Italia per partire con i primi accordi e mettere alla prova, sul campo, il software; il tempo poi ci dirà se l'idea e la sua realizzazione hanno meritato i mesi spesi.
In tutto questo posso solo dire di aver fatto del mio meglio, provando a sfruttare le mie conoscenze, sia accademiche ma soprattutto esterne, per realizzare un buon prodotto che potesse aiutare a risolvere, anche solo in minima parte, il gravissimo e urgente problema dell'inquinamento.

Capitolo 7

Requisiti funzionali

Verranno ora mostrati i principali requisiti funzionali.

- **Invio scontrino**
  - **Introduzione** Si descrive ora il procedimento di invio dello scontrino da parte della cassa e della sua ricezione da parte dell'utente.
  - **Input** La cassa riceve l'ID dell'utente e/o il suo token FCM. L'API pubblica chiamata dalla cassa riceve, in formato JSON, tutto lo scontrino in formato testuale, il prezzo pagato dall'utente, la data di emissione, l'area stimata occupata dallo scontrino se fosse stato emesso in formato cartaceo e il negozio da cui è stato fatto il pagamento.
  - **Processo** Al momento dell'emissione dello scontrino, l'utente mostra il QRcode contenente le proprie informazioni, cosicchè la cassa possa leggerle. A questo punto la cassa invia tutti i dati necessari all'API pubblica gestita dal servizio di backend, il quale legge le informazioni, le con-

valide.
Quindi carica lo scontrino nello spazio di archiviazione online e invia la notifica di avvenuta ricezione scontrino, contenente le informazioni accessorie, all'applicazione.
Quando la notifica arriverà, l'utente potrà decidere se tenere lo scontrino o eliminarlo.

- **Output** Se l'utente deciderà di tenere lo scontrino potrà visualizzarlo in qualsiasi momento.

- **Creazione categoria condivisa**
  - **Introduzione** Con questa procedura si può creare una categoria condivisa così da poter inviare gli scontrini ricevuti agli altri utenti che verranno aggiunti.
  - **Input** Nome della categoria, icona selezionata per la creazione della categoria.
    Email di iscrizione o QRCode dell'utente/degli utenti che si vogliono aggiungere.
  - **Processo** Per la creazione del gruppo basta dirigersi nella pagina delle categorie, premere sul pulsante "+" ed inserire le informazioni desiderate. Quindi verrà creata la voce relativa alla categoria sul database.
    Poi, per aggiungere un altro utente, bisogna andare nella pagina della categoria, premere il pulsante di modifica e a questo punto si può, o inserire la mail dell'utente, oppure si può scansionare il suo QRCode.

  - **Output** L'utente potrà vedere la nuova categoria, denominata come vuole lui, e con i partecipanti che ha deciso di aggiungere.

- **Visualizzazione scontrini per mese**
  - **Introduzione** Procedimento per la visualizzazione degli scontrini in un dato mese.
  - **Input** Mese di cui si vuole vedere l'elenco degli scontrini.
  - **Processo** Ci si dirige nella pagina contenente tutti gli scontrini, "Archivio", e dalla tabella mostrata si sceglie il mese di interesse per l'utente.
    Da qui, verrà applicato un filtro agli scontrini già esistenti e salvati, così da ottenere un elenco, diviso per giorni, di tutti gli scontrini emessi in quel mese, con anche il costo totale, derivato dalla somma dei singoli. Ovviamente gli scontrini saranno consultabili singolarmente premendoci semplicemente sopra.
  - **Output** Elenco contenente tutti gli scontrini emessi nel mese selezionato.

Bibliografia

[1] Dominik Rachoń Aleksandra Konieczna Aleksandra Rutkowska. *Health risk of exposure to Bisphenol A (BPA)*. URL: https://pubmed.ncbi.nlm.nih.gov/25813067/.

[2] *Bucket locations: regions*. URL: https://cloud.google.com/storage/docs/locations%5C#location-r.

[3] Ministero dell'economia e delle finanze. *DECRETO 7 dicembre 2016*. URL: https://www.gazzettaufficiale.it/eli/id/2016/12/29/16A08945/sg. Articolo 1, comma 4.

[4] *FirebaseInstanceIdService: Public Methods*. URL: https://firebase.google.com/docs/reference/android/com/google/firebase/iid/FirebaseInstanceIdService.html%5C#onTokenRefresh().

[5] *Google Cloud Armor Adaptive Protection overview*. URL: https://cloud.google.com/armor/docs/adaptive-protection-overview?authuser=9&_ga=2.15092342.-1940521912.1589290989.

[6] *Hive: benchmark*. URL: https://pub.dev/packages/hive%5C#benchmark.

[7] *Showcase Flutter*. URL: https://flutter.dev/showcase.

[8] Skip the slip. *Environmental & human: health impacts of paper receipts*. URL: https://www.greenamerica.org/sites/default/files/2020-10/Skip%5C%20The%5C%20Slip%5C%20Report%5C%202020%5C%20%5C%28GA%5C%29.pdf. 2020, 3rd edition.

Capitolo 8

Ringraziamenti

Al termine di questo primo percorso universitario vorrei sinceramente ringraziare diverse persone che mi hanno supportato.
Prima di tutte la mia famiglia che mi ha sopportato nei momenti di ansia maggiore. Per la mia prozia a cui devo molti dei miei esami, grazie alle sue innumerevoli preghiere.
Innumerevoli i ringraziamenti che dovrei fare ai miei amici, per dirne alcuni: Enri perchè c'è sempre stato, Marco con cui ho condiviso la maggior parte dei giorni in questi anni, Leo perchè è stato ed è una presenza costante su cui posso sempre far affidamento, Seif con cui ho avuto modo di confrontarmi diverse volte e che è riuscito a stimolare sempre il mio intelletto, la Mati per la sua amicizia sincera e ironica che è riuscita a tirarmi sempre su di morale quando ce n'era bisogno, la Giada a cui devo circa venti punti del mio punteggio di laurea grazie ai suoi appunti, tutto il gruppo della parrocchia, gli amici della diocesi, compagni di università, amici storici e tutti gli altri amici non così facilmente raggruppabili dentro delle categorie.

Voglio inoltre ringraziare Simone Leoni per l'idea della start-up, per aver riposto in me la sua fiducia e per l'energia che ha messo e che sta mettendo nel progetto; a questo si uniscono anche i ringraziamenti ad Andy e Marco per la loro continua e preziosa disponibilità, a Filippo Vidoni per la carica e la passione che ha messo fin da subito nel progetto, e a tutti gli altri collaboratori come Gabri, Edo e Michi che ci hanno accompagnato con entusiasmo.
Ringrazio anche il mio relatore Nicola Bicocchi, che ha sempre dimostrato una grande affabilità, competenza e sincerità prima come insegnante, poi come tutor di questo progetto.